

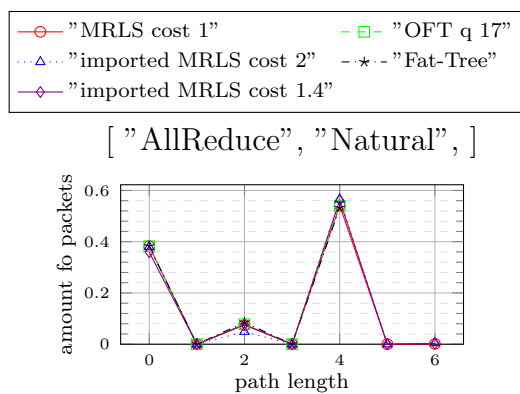


Figure 1: X: ["AllReduce", "Natural",]

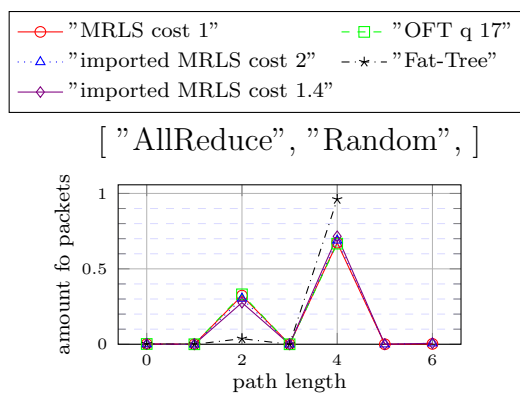


Figure 2: X: ["AllReduce", "Random",]


```

    CartesianEmbedding {
      source_sides: [ 8192 ],
      destination_sides: [ 11664 ]}],
    middle_sizes: [ 8192, 11664 ],
    legend_name: "Natural"},
  Composition {
    patterns: [
      RandomPermutation { legend_name: "Random" },
      CartesianEmbedding {
        source_sides: [ 8192 ],
        destination_sides: [ 11664 ]}],
      middle_sizes: [ 8192, 11664 ],
      legend_name: "Random"}],
    application: AllReduce { tasks: 8192, data_size: 65536, legend_name: "AllReduce" },
    legend_name: "AllReduce"}],
  TrafficMap {
    tasks: 11340,
    map: ![
      Composition {
        patterns: [
          Identity { legend_name: "Natural" },
          CartesianEmbedding {
            source_sides: [ 8192 ],
            destination_sides: [ 11340 ]}],
            middle_sizes: [ 8192, 11340 ],
            legend_name: "Natural"},
          Composition {
            patterns: [
              RandomPermutation { legend_name: "Random" },
              CartesianEmbedding {
                source_sides: [ 8192 ],
                destination_sides: [ 11340 ]}],
                middle_sizes: [ 8192, 11340 ],
                legend_name: "Random"}],
            application: ![
              AllReduce { tasks: 8192, data_size: 65536, legend_name: "AllReduce" }],
              legend_name: "AllReduce"}]],
    maximum_packet_size: 16,
    router: InputOutput {
      virtual_channels: topo! [ 4, 4, 4, 2, 4 ],
      virtual_channel_policies: topo! [
        [
          MapHop {
            hop_to_policy: [
              ArgumentVC {
                allowed: [ 0 ]},
              ArgumentVC {
                allowed: [ 0 ]},
              ArgumentVC {
                allowed: [ 1 ]},
              ArgumentVC {
                allowed: [ 1 ]},
              ArgumentVC {
                allowed: [ 2 ]},
              ArgumentVC {
                allowed: [ 2 ]},
              ArgumentVC {
                allowed: [ 3 ]},
              ArgumentVC {
                allowed: [ 3 ]}]},
            VecLabel {
              label_vector: [ 0, 64, 80 ]},
              OccupancyFunction { label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
                use_neighbour_space: true, aggregate: true },
              OccupancyFunction { label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
                use_neighbour_space: true, aggregate: false },
              LowestLabel,
              EnforceFlowControl,
              Random],
            [
              MapHop {
                hop_to_policy: [
                  ArgumentVC {
                    allowed: [ 0 ]},
                  ArgumentVC {
                    allowed: [ 0 ]},
                  ArgumentVC {
                    allowed: [ 1 ]},
                  ArgumentVC {
                    allowed: [ 1 ]},
                  ArgumentVC {
                    allowed: [ 2 ]},
                  ArgumentVC {
                    allowed: [ 2 ]},
                  ArgumentVC {
                    allowed: [ 3 ]},
                  ArgumentVC {
                    allowed: [ 3 ]}]},
                VecLabel {
                  label_vector: [ 0, 64, 80 ]},
                  OccupancyFunction { label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
                    use_neighbour_space: true, aggregate: true },
                  OccupancyFunction { label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
                    use_neighbour_space: true, aggregate: false },
                  LowestLabel,
                  EnforceFlowControl,
                  Random],
                [
                  MapHop {
                    hop_to_policy: [
                      ArgumentVC {
                        allowed: [ 0 ]},
                      ArgumentVC {
                        allowed: [ 0 ]},
                      ArgumentVC {
                        allowed: [ 1 ]}

```

```

    ArgumentVC{
      allowed: [ 1 ]},
    ArgumentVC{
      allowed: [ 2 ]},
    ArgumentVC{
      allowed: [ 2 ]},
    ArgumentVC{
      allowed: [ 3 ]},
    ArgumentVC{
      allowed: [ 3 ]}],
  VecLabel{
    label_vector: [ 0, 64, 80 ]},
  OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
use_neighbour_space: true, aggregate: true },
  OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
use_neighbour_space: true, aggregate: false },
  LowestLabel,
  EnforceFlowControl,
  Random],
[
  OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
use_neighbour_space: true, aggregate: true },
  OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
use_neighbour_space: true, aggregate: false },
  LowestLabel,
  EnforceFlowControl,
  Random],
[
  MapHop{
    hop_to_policy: [
      ArgumentVC{
        allowed: [ 0 ]},
      ArgumentVC{
        allowed: [ 0 ]},
      ArgumentVC{
        allowed: [ 1 ]},
      ArgumentVC{
        allowed: [ 1 ]},
      ArgumentVC{
        allowed: [ 1 ]},
      ArgumentVC{
        allowed: [ 2 ]},
      ArgumentVC{
        allowed: [ 2 ]},
      ArgumentVC{
        allowed: [ 3 ]},
      ArgumentVC{
        allowed: [ 3 ]}],
    VecLabel{
      label_vector: [ 0, 64, 80 ]},
    OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
use_neighbour_space: true, aggregate: true },
    OccupancyFunction{ label_coefficient: 1, occupancy_coefficient: 1, product_coefficient: 0, constant_coefficient: 0, use_internal_space: true,
use_neighbour_space: true, aggregate: false },
    LowestLabel,
    EnforceFlowControl,
    Random]],
  allocator: Random,
  buffer_size: 128,
  bubble: false,
  flit_size: 16,
  intransit_priority: false,
  allow_request_busy_port: true,
  output_buffer_size: 64,
  crossbar_frequency_divisor: 1,
  crossbar_delay: 2},
routing: topo![
  Polarized{ include_labels: true, strong: false, legend_name: "Polarized" },
  Polarized{ include_labels: true, strong: false, legend_name: "Polarized" },
  Polarized{ include_labels: true, strong: false, legend_name: "Polarized" },
  UpDown{ legend_name: "minimal routing" },
  Polarized{ include_labels: true, strong: false, legend_name: "Polarized" }],
link_classes: [
  LinkClass{ delay: 2 },
  LinkClass{ delay: 2 },
  LinkClass{ delay: 2 }],
launch_configurations: [
  Slurm{ job_pack_size: 1, time: "3-00:00:00" }]]

```